

MATERIAL DE AULA - TURMA POO

Python POO - Dia 5

Aquecimento e retomada do RPG

| *“Primeiro recuperamos as peças. Depois montamos o jogo.”*

PARTE 1

Mapa do que será recuperado

Antes de programar, olhe esse caminho. Cada peça se apoia na anterior, e é essa sequência que vamos percorrer hoje, do começo ao fim.

Classe → o molde que descreve como um personagem é.

Objeto → um personagem específico, criado a partir da classe.

Estado → os valores atuais desse objeto: vida, ataque, poções.

Método → uma ação que o objeto sabe executar.

Interação → um objeto chamando um método de outro objeto.

Turno → uma rodada: jogador age, depois o inimigo age.

Batalha → turnos repetidos em um `while`, até alguém chegar a zero de vida.

Dica: Pular direto para o `while` da batalha sem confirmar as peças de trás costuma gerar erros difíceis de achar depois. Hoje testamos cada peça isoladamente antes de juntar tudo.

A pausa de uma semana não apagou o que vocês construíram, só deixou o raciocínio um pouco frio. O objetivo de hoje não é aprender coisa nova, é destravar o que já está aí.

PARTE 2

Aquecimento sem computador

Sem abrir o editor ainda. Responda com suas palavras, não precisa ser formal, precisa fazer sentido pra você.

1. Qual é a diferença entre **classe** e **objeto**?

2. Para que serve o `__init__`?

3. O que é o `self`?

4. Qual é a diferença entre um **atributo** e um **método**?

5. Como você chama um método de um objeto que já existe?

Dica: Escreva o que você lembra, mesmo incompleto. A ideia é ver o que já está sólido e o que precisa de reforço, não tirar nota.

EXERCÍCIO 1

Complete a classe

Esta é a classe que vamos usar o dia todo. Complete o que falta.

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        # completar

    def mostrar_status(self):
        # completar
```

Complete:

- os três atributos dentro do `__init__`;
- `mostrar_status()`, imprimindo nome, vida e ataque do personagem.

Depois, crie:

- um jogador chamado **Thoric**, com 30 de vida e 8 de ataque;
- um inimigo chamado **Goblin**, com 18 de vida e 5 de ataque;
- mostre o estado dos dois com `mostrar_status()`.

Chamada	Sua previsão de saída
<code>thoric.mostrar_status()</code>	
<code>goblin.mostrar_status()</code>	

Antes de executar: Preencha a coluna da direita com o que você acha que vai aparecer. Só depois rode o código e compare.

EXERCÍCIO 2

Encontre e corrija os erros

O código abaixo tem **quatro erros**. Nenhum deles é raro, são os erros mais comuns de quem está começando com classes. Encontre, explique e corrija cada um.

```
class Personagem:
    def __init__(nome, vida, ataque):
        nome = nome
        self.vida = vida
        self.ataque = ataque

    def mostrar_status(self):
        print(self.nome, "tem", self.vida, "de vida")

thoric = Personagem("Thoric", 30)
thoric.mostrarstatus()
```

Erro	Onde está	Por que está errado	Correção
1			
2			
3			
4			

Dica: Todo método de instância, incluindo o `__init__`, precisa de `self` como primeiro parâmetro. E todo atributo que deve "grudar" no objeto precisa do prefixo `self.`

EXERCÍCIO 3

Receber dano

Volte à sua classe `Personagem` (já corrigida) e implemente:

```
def receber_dano(self, quantidade):  
    # completar
```

Regras:

- subtrair `quantidade` da vida do personagem;
- a vida **nunca** pode ficar negativa, no mínimo zero;
- mostrar quanto dano foi recebido;
- mostrar a vida restante depois do dano.

Teste manualmente, prevendo o resultado antes de rodar:

Chamada	Vida antes	Sua previsão de vida depois
<code>thoric.receber_dano(12)</code>	30	
<code>thoric.receber_dano(25)</code>	(o valor que sobrou)	

Atenção ao segundo teste: $30 - 12 - 25$ dá um número negativo se você só subtrair. O que a regra pede que aconteça nesse caso?

EXERCÍCIO 4

Um objeto ataca outro

Agora implemente o método que conecta os dois personagens:

```
def atacar(self, alvo):  
    # completar
```

O método deve chamar:

```
alvo.receber_dano(self.ataque)
```

`self` = quem está atacando. `alvo` = quem está recebendo o ataque.

Em `jogador.atacar(inimigo)`: `self` é `jogador`, `alvo` é `inimigo`. O método chama `alvo.receber_dano(self.ataque)`, ou seja, a vida de `inimigo` é alterada usando o ataque de `jogador`.

- Em `jogador.atacar(inimigo)`, qual objeto será `self`?

- Qual objeto será `alvo`?

- De qual objeto vem o valor usado como dano?

- Qual objeto vai ter a vida alterada?

EXERCÍCIO 5

Batalha manual

Antes de automatizar qualquer coisa com `while`, confirme que os objetos conversam direito entre si. Execute isto **manualmente**, linha por linha:

```
jogador.atacar(inimigo)
inimigo.mostrar_status()

inimigo.atacar(jogador)
jogador.mostrar_status()
```

1. Antes de rodar, preveja a vida final dos dois personagens.
2. Execute e confira o resultado real.
3. Compare sua previsão com o que aconteceu.
4. Escreva mais uma rodada manual (mais um ataque de cada lado) e repita o processo.

	Vida prevista	Vida real
jogador		
inimigo		

Dica: Se um erro aparecer aqui, você sabe exatamente qual método revisar. Dentro de um `while` automático, o mesmo erro fica muito mais difícil de encontrar.

EXERCÍCIO 6

Escolha de ação

Complete a função que organiza o turno do jogador:

```
def turno_do_jogador(jogador, inimigo):  
    print("1 - Atacar")  
    print("2 - Defender")  
    print("3 - Usar poção")  
  
    escolha = input("Escolha: ")  
  
    # completar com if/elif/else
```

Relacione cada opção do menu a um método que já existe no seu personagem:

Opção digitada	Método chamado
"1"	
"2"	
"3"	
qualquer outra coisa	(não deve travar o jogo)

Dica: `defender()` e `usar_pocao()` vocês já construíram antes da pausa, na versão completa da classe (com os atributos `defesa` e `pocoas`). Se ainda estiverem no seu arquivo, é só reaproveitar. Se não estiverem, peça a versão pronta ao professor. Hoje o foco é o turno, não reescrever esses métodos.

Erro comum: `escolha` vem do `input()`, então é sempre uma string. Compare com `"1"`, `"2"`, `"3"`, e não com os números `1`, `2`, `3`.

`turno_do_jogador` é uma *função* que organiza a rodada. Ela não faz nada sozinha, ela chama os *métodos* que já existem na classe `Personagem`. Não misture os dois papéis.

EXERCÍCIO 7

Complete o while da batalha

Agora junte tudo em um laço. Complete as duas lacunas e a mensagem final:

```
while _____:
    jogador.mostrar_status()
    inimigo.mostrar_status()

    turno_do_jogador(jogador, inimigo)

    if _____:
        inimigo.atacar(jogador)

# depois do laço: mensagem de vitória ou derrota
```

Complete:

- a condição do `while`: o jogo continua enquanto o quê?
- a verificação que impede um inimigo **já derrotado** de atacar de volta;
- a mensagem final, dizendo quem venceu.

Fluxo de uma rodada: mostrar estado dos dois, jogador escolhe a ação, jogador age, checar se o inimigo ainda está vivo, inimigo age, checar se alguém chegou a zero de vida, repetir ou encerrar.

EXERCÍCIO 8

Pequeno avanço

Sua batalha já funciona do começo ao fim. Agora escolha **apenas uma** das opções abaixo para deixá-la um pouco mais interessante.

A) Dano variável. Use `random.randint()` para o dano de `atacar()` variar um pouco a cada vez, em vez de ser sempre igual a `self.ataque`. *Dica: pense em uma faixa em torno de `self.ataque`, um pouco abaixo e um pouco acima.*

B) Chance de erro. Antes de causar dano, role um dado de 1 a 20. Abaixo de um certo valor, o ataque erra. *Dica: se o ataque errar, `receber_dano()` não deve ser chamado, só uma mensagem avisando o erro.*

C) Ataque crítico. Role um dado de 1 a 20. Se der exatamente 20, o dano é maior que o normal. *Dica: você pode dobrar o dano só nesse caso, pense em como comparar a rolagem com o número 20.*

D) Inimigo imprevisível. Em vez de o inimigo sempre atacar, sorteie entre atacar ou defender no turno dele. *Dica: um `random.randint(1, 2)` pode decidir entre as duas opções.*

Escolha uma, implemente, teste algumas vezes (o resultado deve mudar entre uma execução e outra) e só então mostre ao professor. As outras três opções ficam para depois.

DESAFIO OPCIONAL

Dois inimigos

Só para quem terminar cedo. Crie uma lista com dois inimigos e enfrente-os em sequência, um `for` por fora do `while` da batalha.

Regras:

- o jogador mantém a vida atual de um combate para o outro, não reinicia;
- as poções não são recuperadas automaticamente entre combates;
- se o jogador morrer no primeiro combate, o segundo **não começa**;
- mostre o nome do inimigo antes de cada batalha começar.

Dica: O laço `while` que você já tem do Exercício 7 continua igual, só passa a rodar uma vez para cada inimigo da lista, dentro de um `for`.

FECHAMENTO

Checklist do aluno

- ☐ Criei a classe.
 - ☐ Usei `__init__` e `self`.
 - ☐ Criei pelo menos dois objetos.
 - ☐ Fiz um objeto atacar outro.
 - ☐ Impedi a vida de ficar negativa.
 - ☐ Organizei a escolha do jogador.
 - ☐ Completei o `while` da batalha.
 - ☐ O jogo anuncia vitória ou derrota.
 - ☐ Consigo explicar quem são `self` e `alvo`.
-

FECHAMENTO

Perguntas de saída

1. O que `self` representa?

2. Por que testar os métodos manualmente antes do `while`?

3. O que faz o laço da batalha terminar?

Hoje o objetivo não era construir um RPG completo, era recuperar o fluxo e terminar com uma batalha funcional.